

Resolving Problems by Searching I

AIF-MSc on AI / 2026-2027

David Chaves-Fraga

University of Santiago de Compostela (Spain)

david.chaves@usc.es



Singular Research Center on
Intelligent technologies

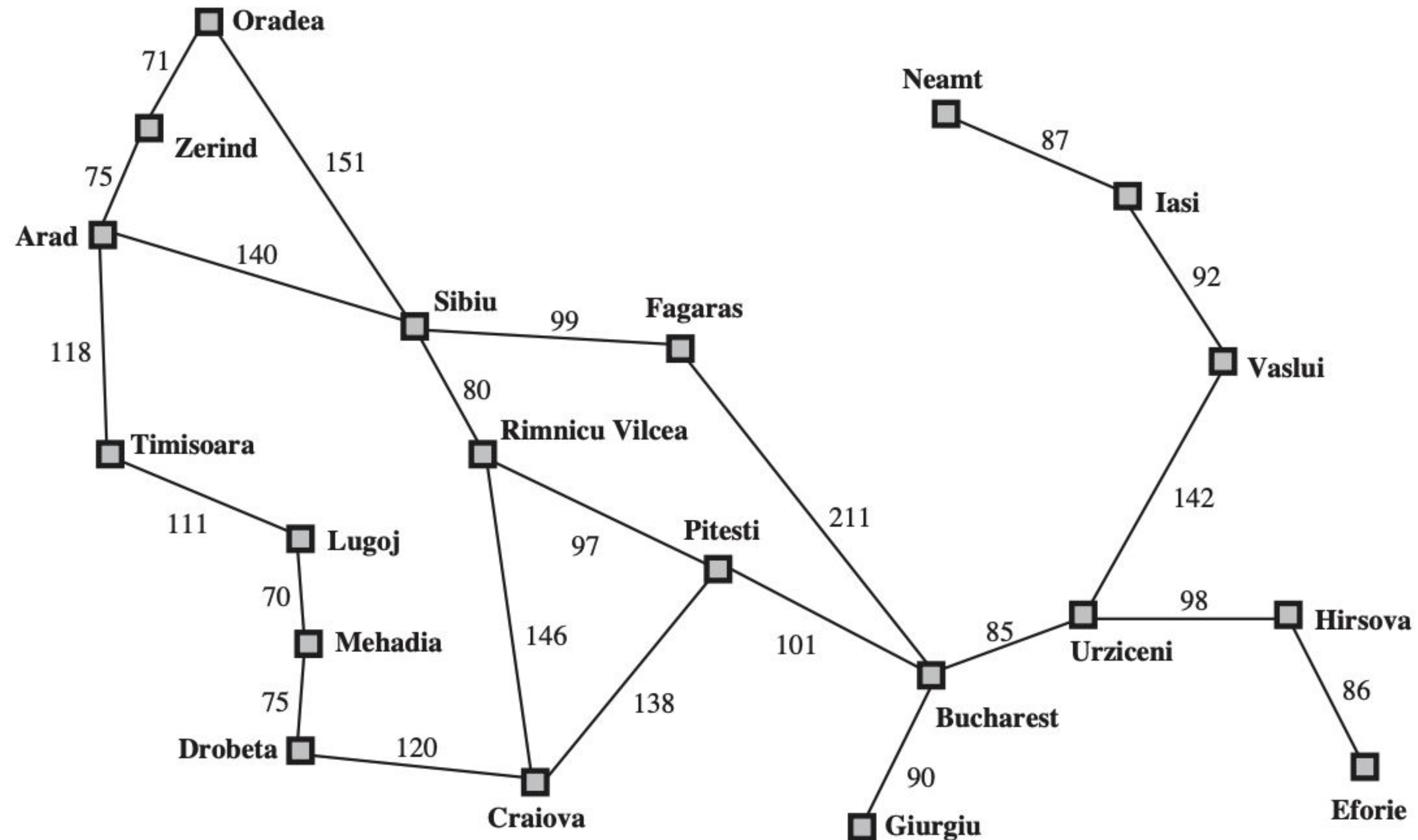


Contents

Problem solving in AI

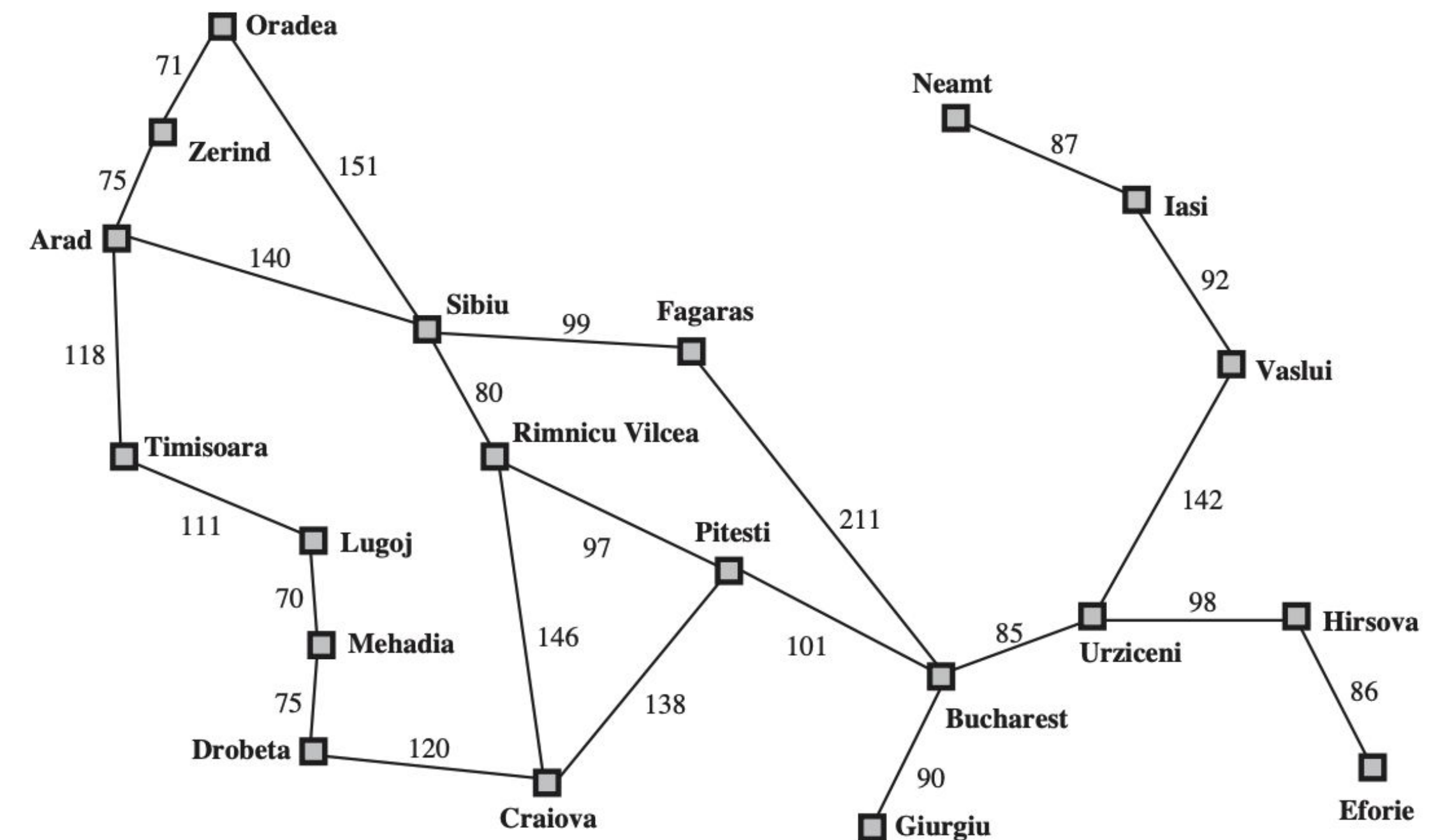
- Search algorithms
- Uninformed search algorithms
- Informed (heuristic) search strategies
- Heuristic functions
- Search for Constraint satisfaction problems

Example of a problem - Routes in Romania



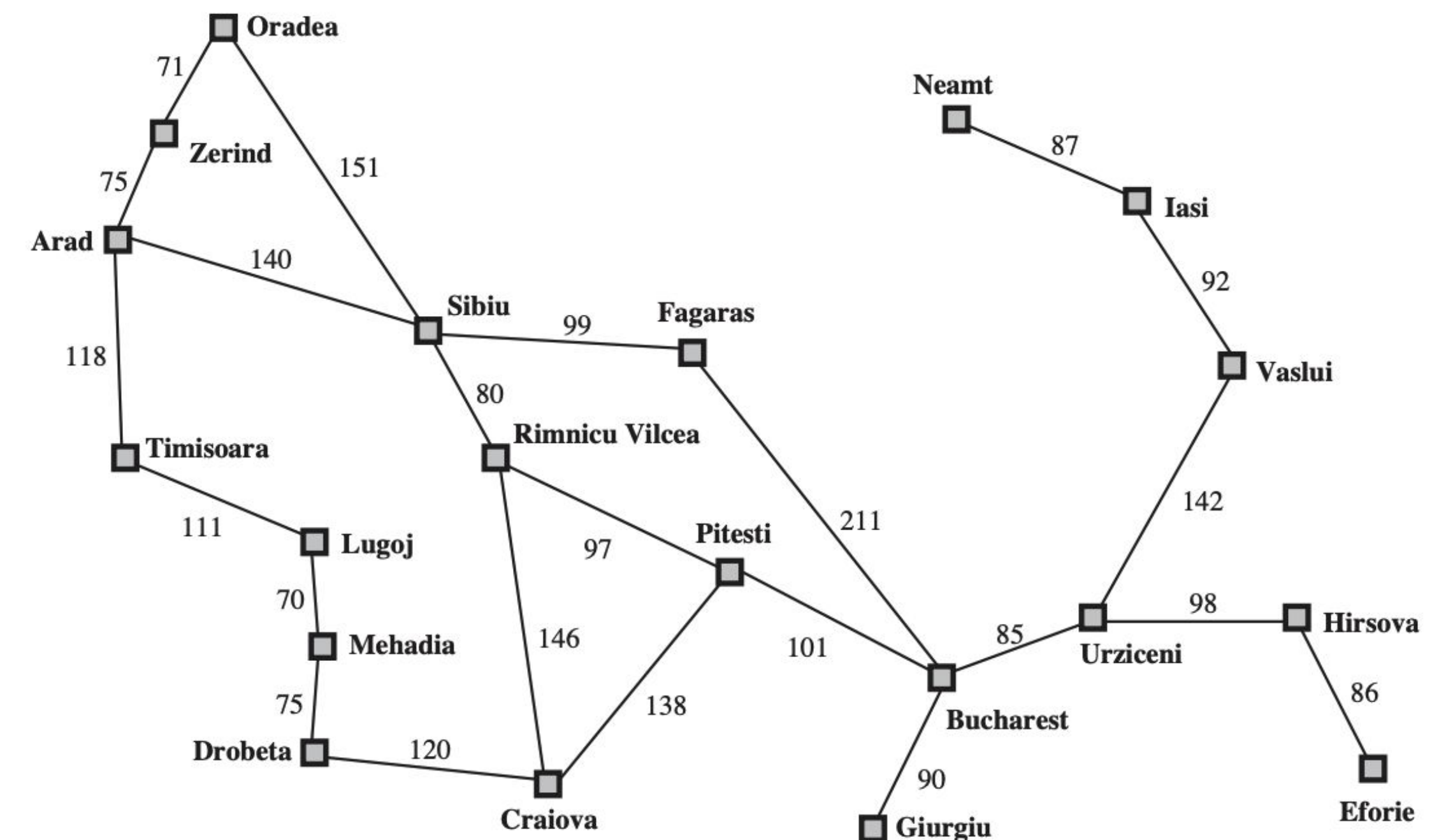
Example of a problem - Routes in Rumania

- **Goal:** What is the best path to reach Bucarest?
- **Problem:** states and actions
 - States: ??
 - Actions: ??
- **Search:** sequences of action until the goal is reached (i.e. a solution)
- **Execution:** actually performing the action of the solution to solve the problem



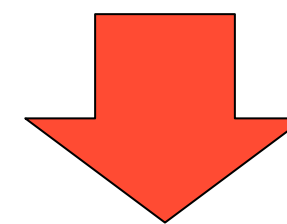
Example of a problem - Routes in Rumania

- **Goal:** What is the best path to reach Bucarest?
- **Problem:** states and actions
 - *States: Cities*
 - *Actions: Travelling to an adjacent city*
- **Search:** sequences of action until the goal is reached (i.e. a solution)
- **Execution:** actually performing the action of the solution to solve the problem



Search Algorithms - Formal definition

- **State space:** set of possible states that the environment can be in
- **Initial state** that the agent starts in
- One or more **goal states (+ corresponding goal tests)**
- A description of the possible **actions** available to the agent
- **Transition model:** description of what each action does
- **Cost function:** numeric cost of applying one action a in state s to reach state s' .
- **Sequence of actions:** path (and path cost)



Single data structure as input for a problem-solving algorithm

Search Algorithms

- **Solution:** path from the initial state to a goal state
- **Optimal solution:** lowest path cost among all solutions
- The state space can be represented **as a directed network or graph:**
 - vertices/nodes are states
 - directed edges between states are actions

Basic Algorithm for a Problem Solving Agent

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

persistent: *seq*, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null

problem, a problem formulation

state $\leftarrow$ UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal $\leftarrow$ FORMULATE-GOAL(*state*)

problem $\leftarrow$ FORMULATE-PROBLEM(*state*, *goal*)

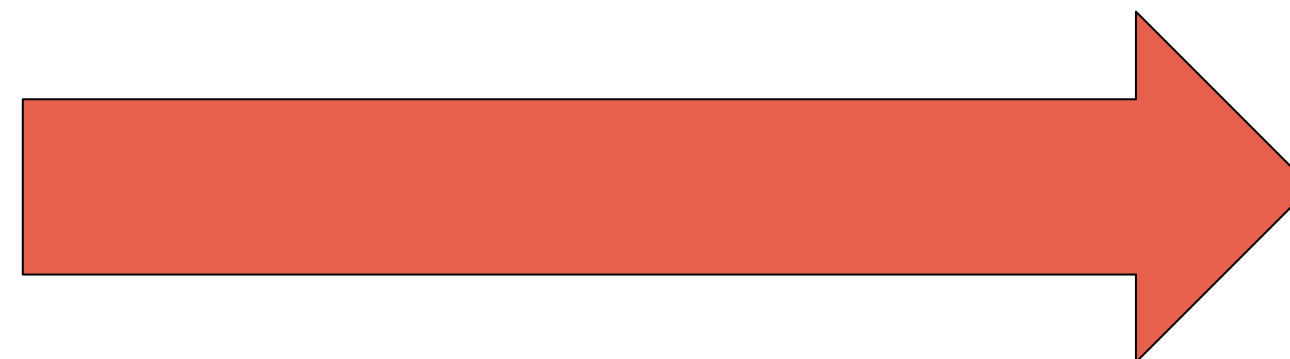
seq $\leftarrow$ SEARCH(*problem*)

if *seq* = *failure* **then return** a null action

action $\leftarrow$ FIRST(*seq*)

seq $\leftarrow$ REST(*seq*)

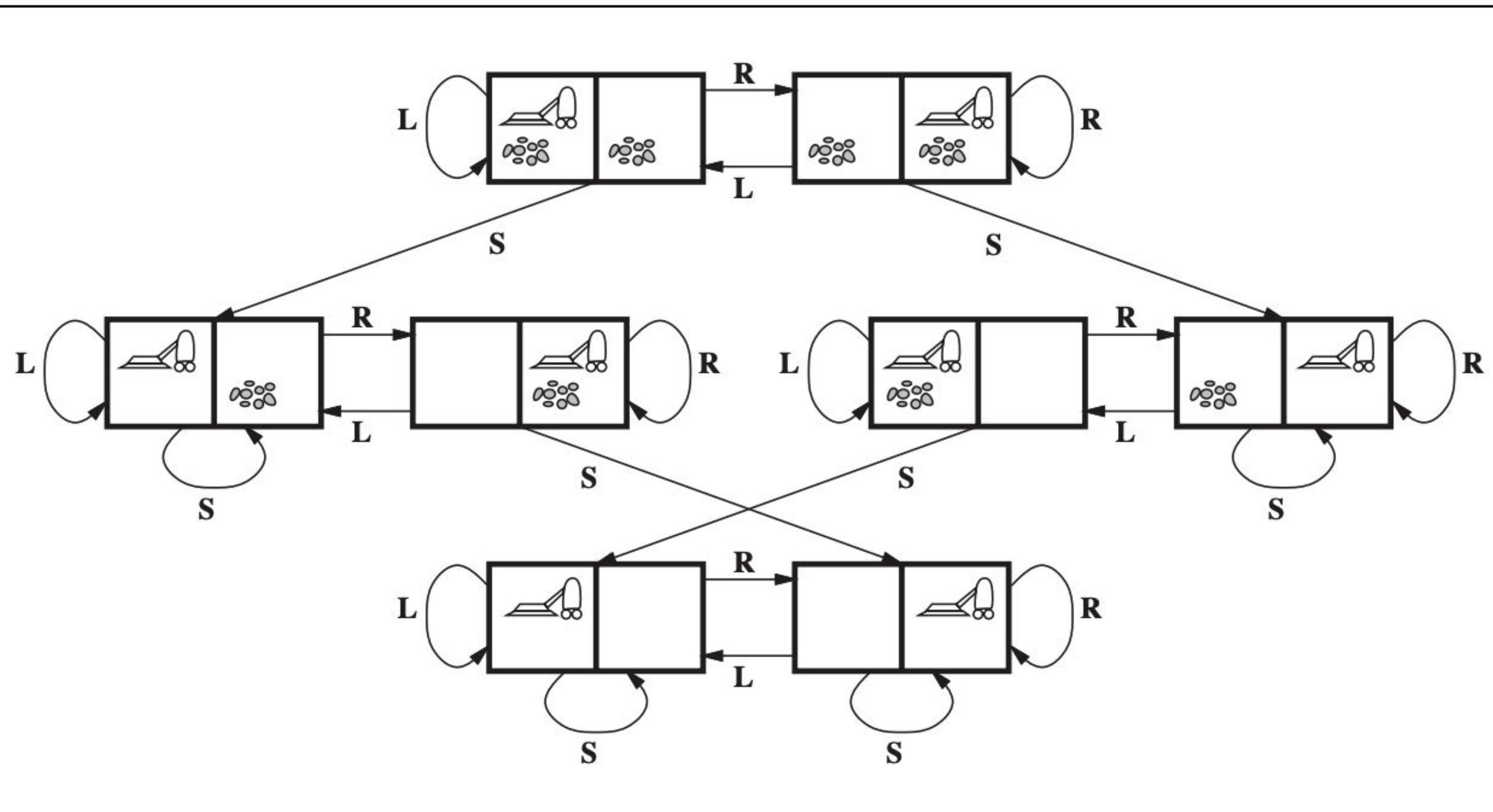
return *action*



Assuming a simple queue



Search Algorithms - Toy examples



The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

2-cells: $2 \cdot 2 \cdot 2$ states ($n \cdot 2^n$)

Actions: L, R, S

Which is the transition model??

Which is/are the Goal state/s??

Action cost: 1

Search Algorithms - Another toy example

7	2	4
5		6
8	3	1

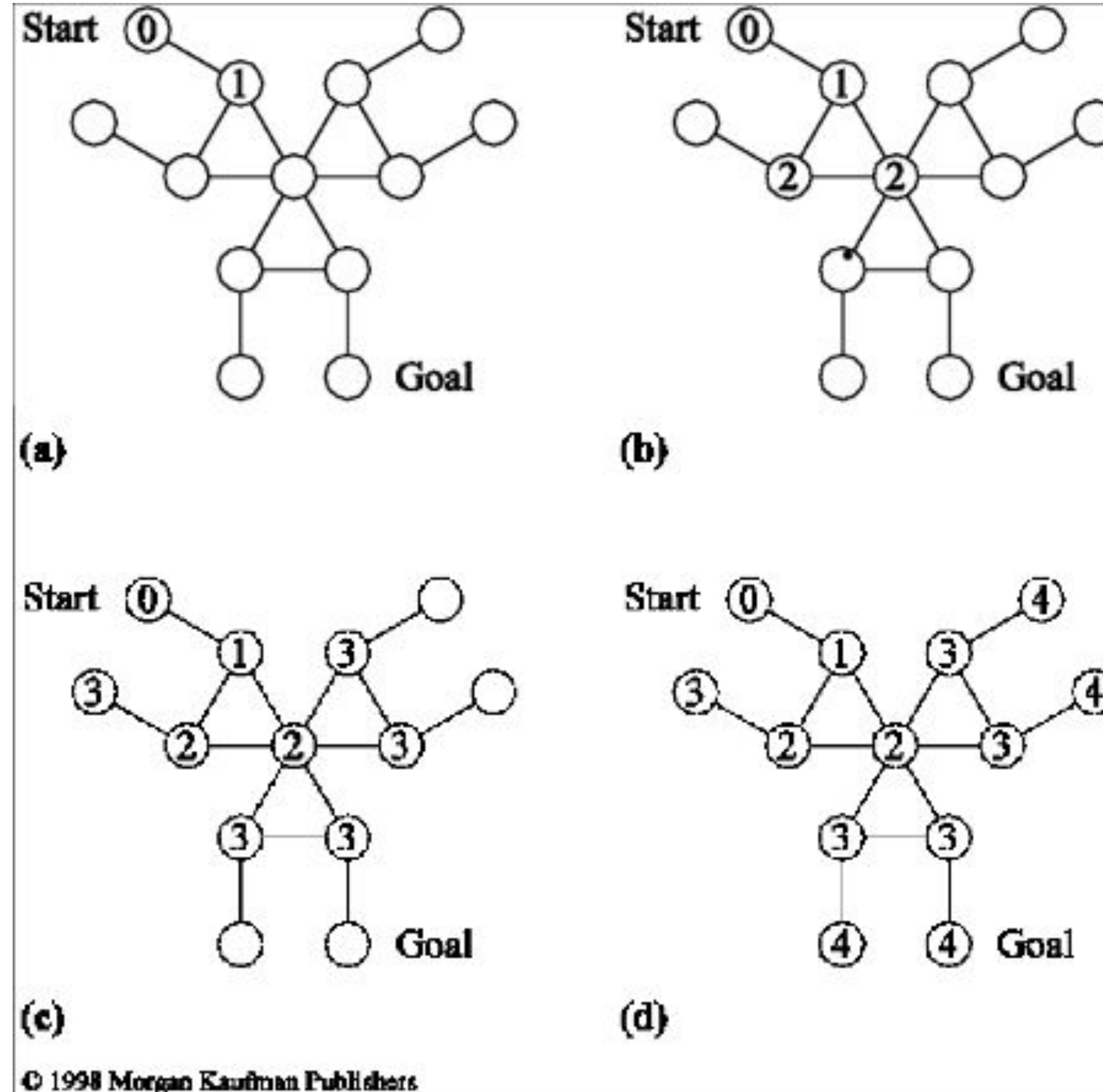
Start State

	1	2
3	4	5
6	7	8

Goal State

- **States:** a representation of the tiles position (vector)
- **Operators:** allowed movements (for convenience, "movement" of the empty space) $^{\wedge}, v, >, <$
- **Goal state:** See figure
- **Action cost:** 1

Search Algorithms - Blocks with cost



Search Algorithms - Real Problems

- Route-finding problem
- Touring problems: TSP
- VLSI layout
- Robot navigation
- Automatic Assembly sequencing
 - Protein design

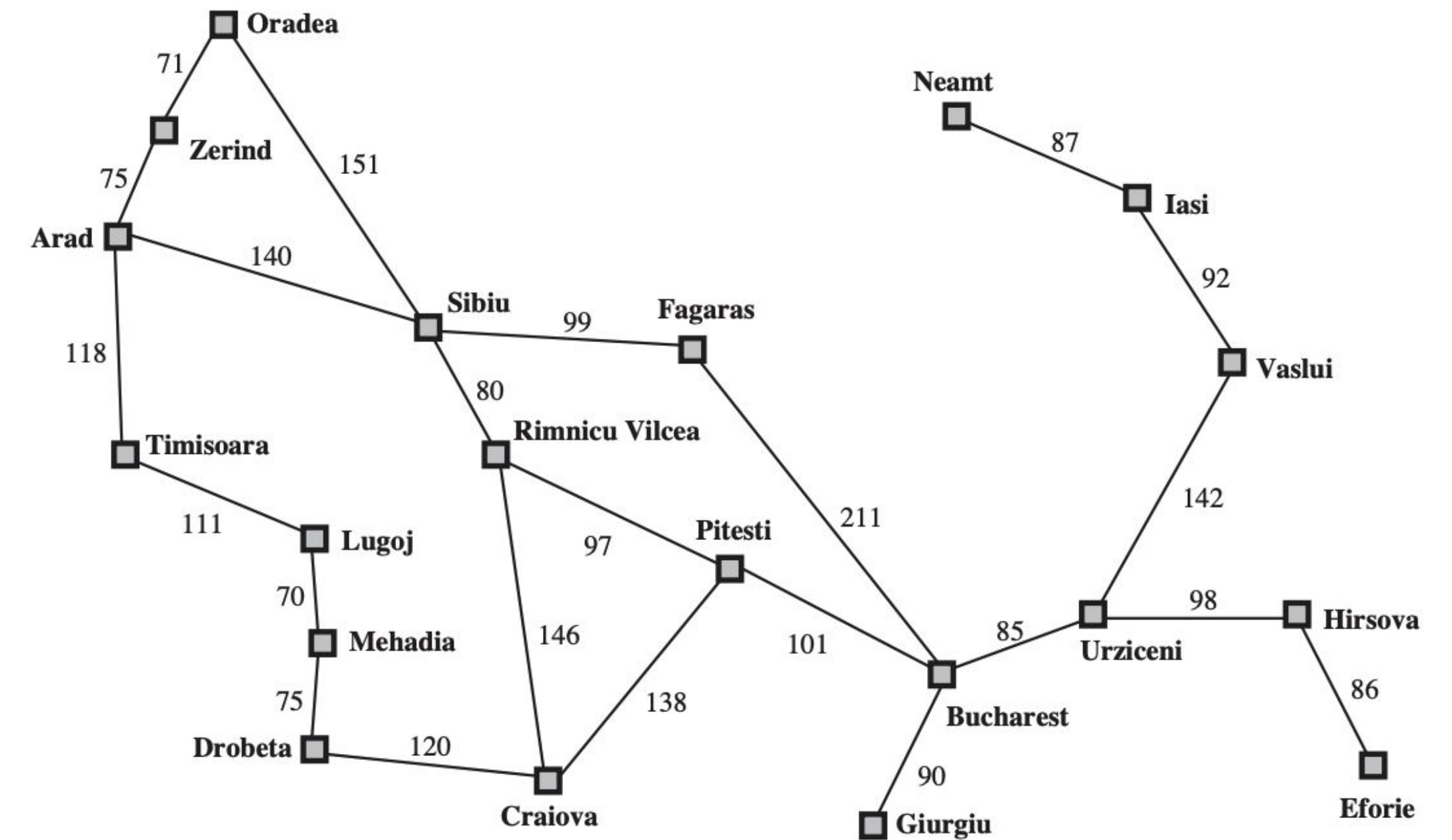
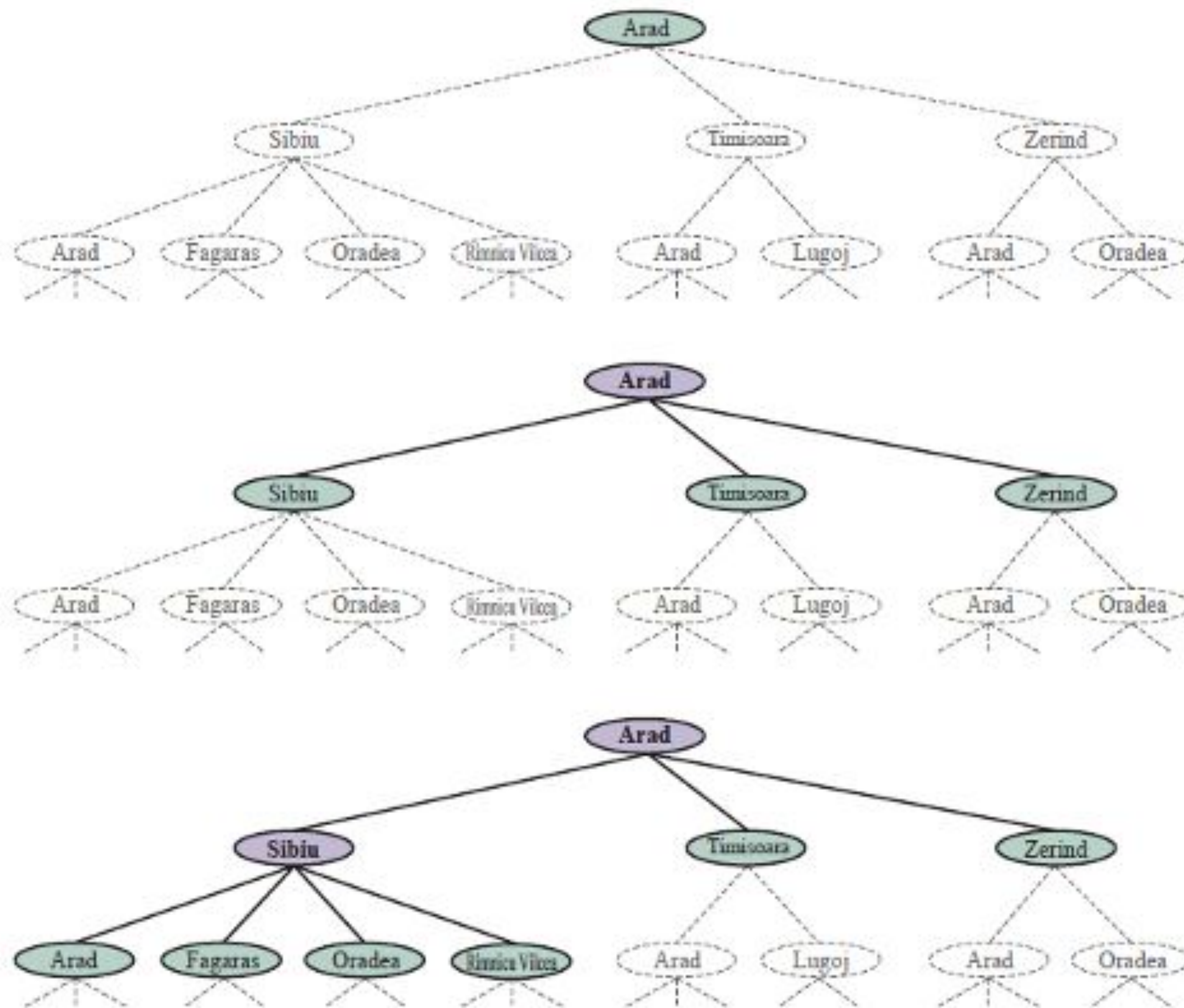
Exercise 1: find, describe briefly and propose a simple model for a real search problem



Search Algorithms – How to search

Method	Description	Solution type
Precise	Specific method for a problem which provides an optimal solution	Exact
Heuristic	Problem resolution problem involving specific knowledge about the domain, aiming to provide an approximate solution or increase the efficiency of a solution	Approximate
Metaheuristics	Improve heuristics, by generalization and achieving efficiency improvements. Generally, they use strategies which make search more diverse, avoiding local minima and more intense in promising zones of the search space	Approximate

Search Algorithms



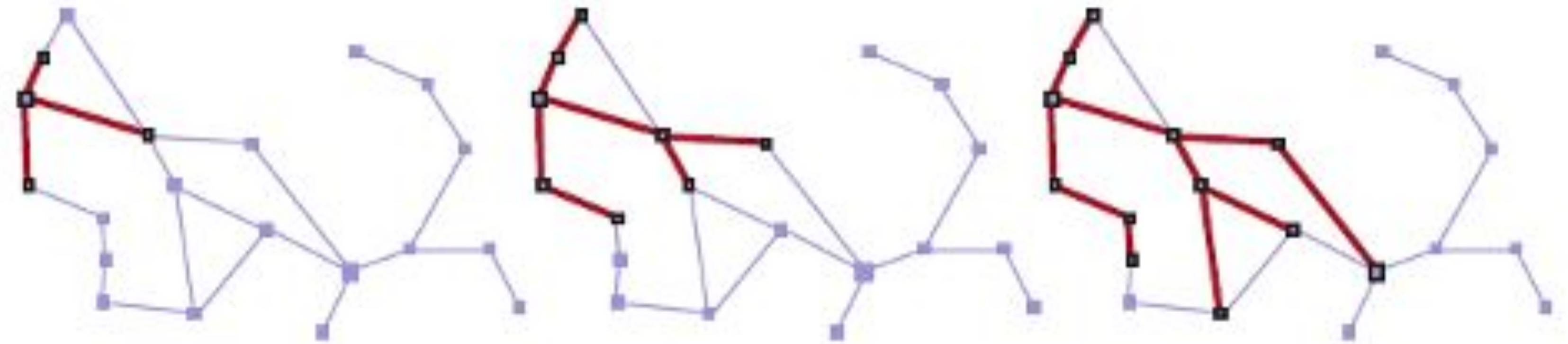
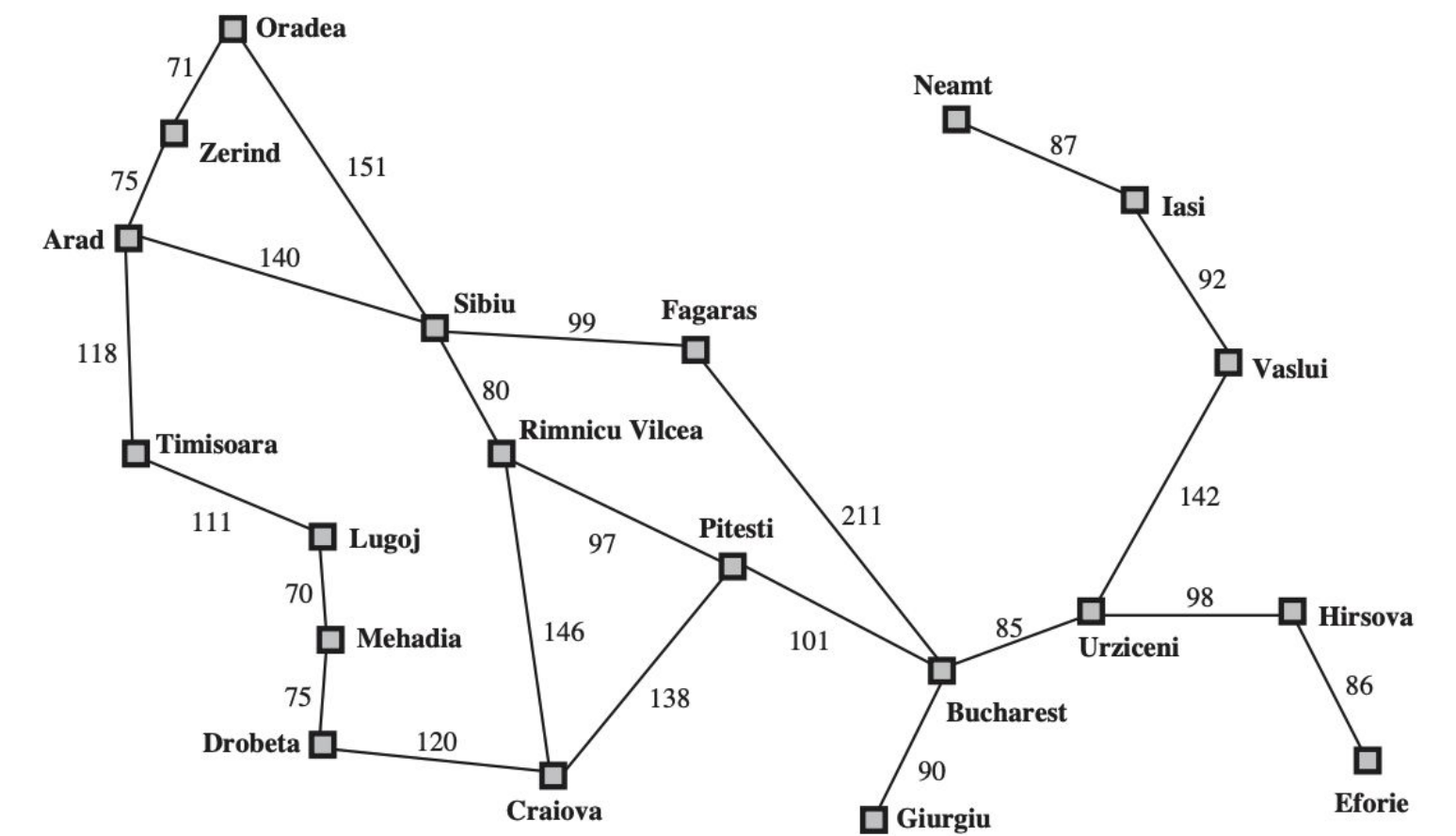
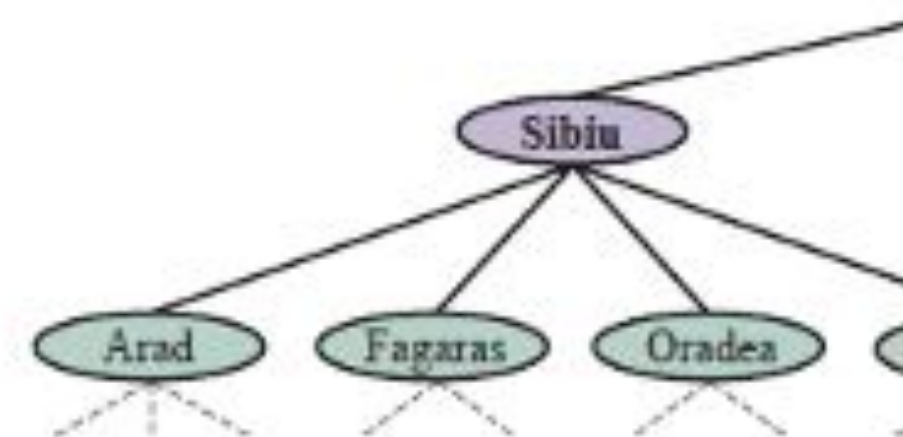
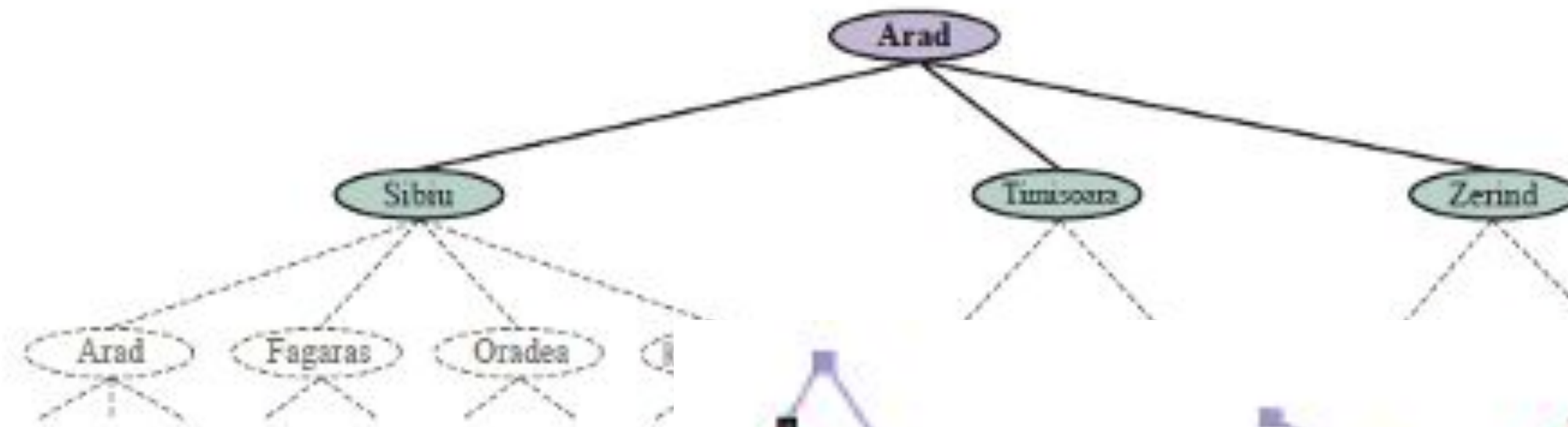
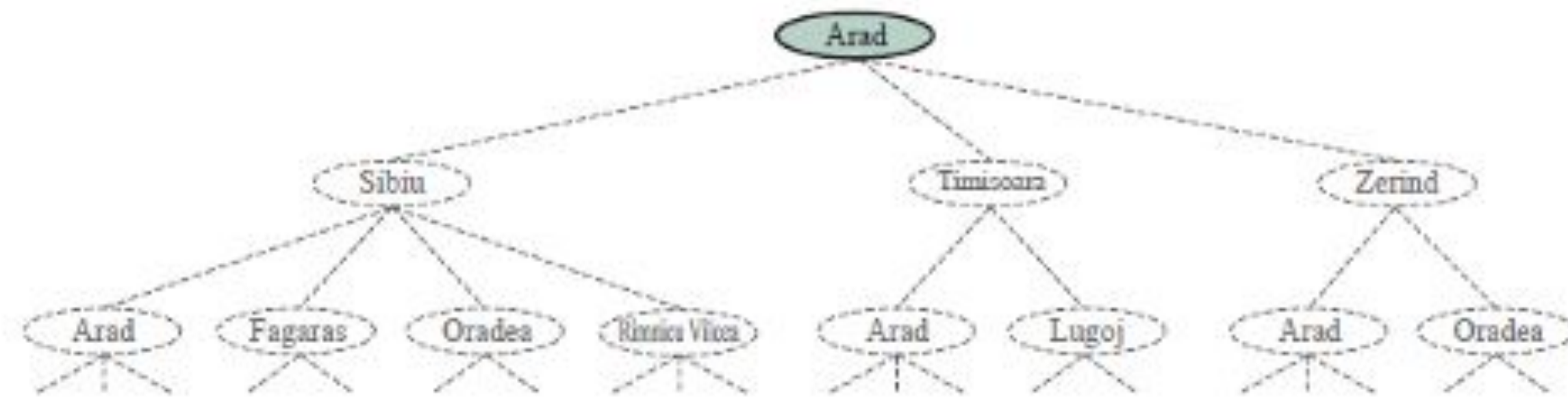
Search tree != state space

- Node: state
- Edge: actions

Created nodes vs Expanded nodes



Search Tree

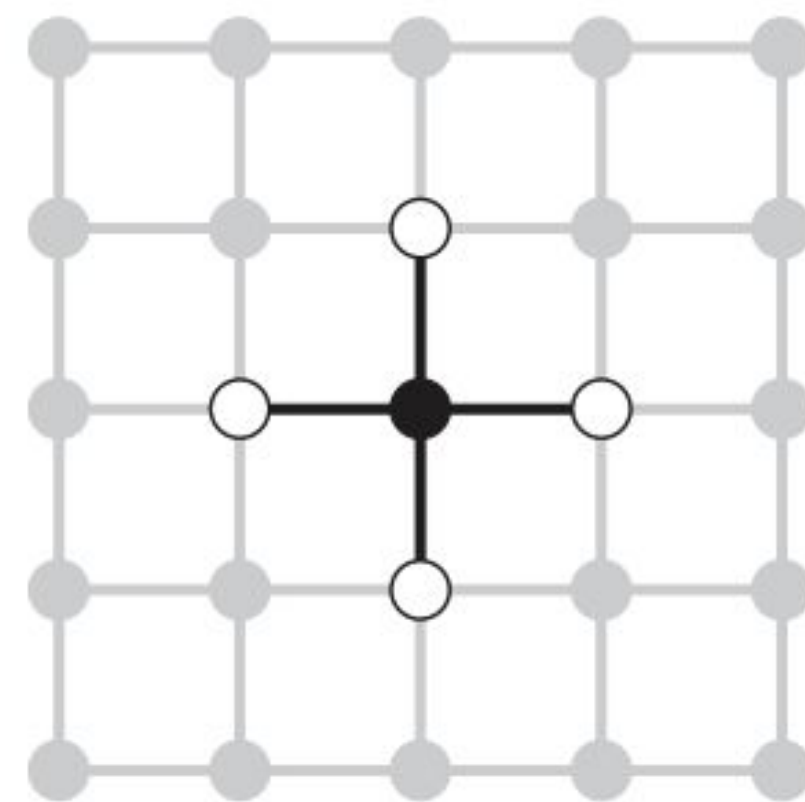


Search Tree

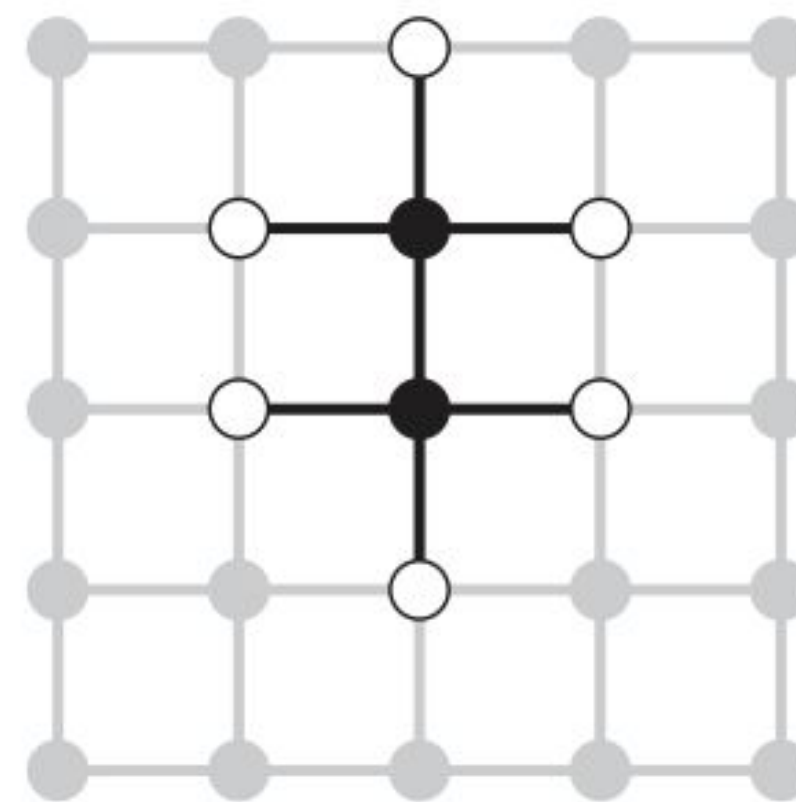
Frontier: nodes reached but not expanded (white nodes)

Interior: nodes expanded (black nodes)

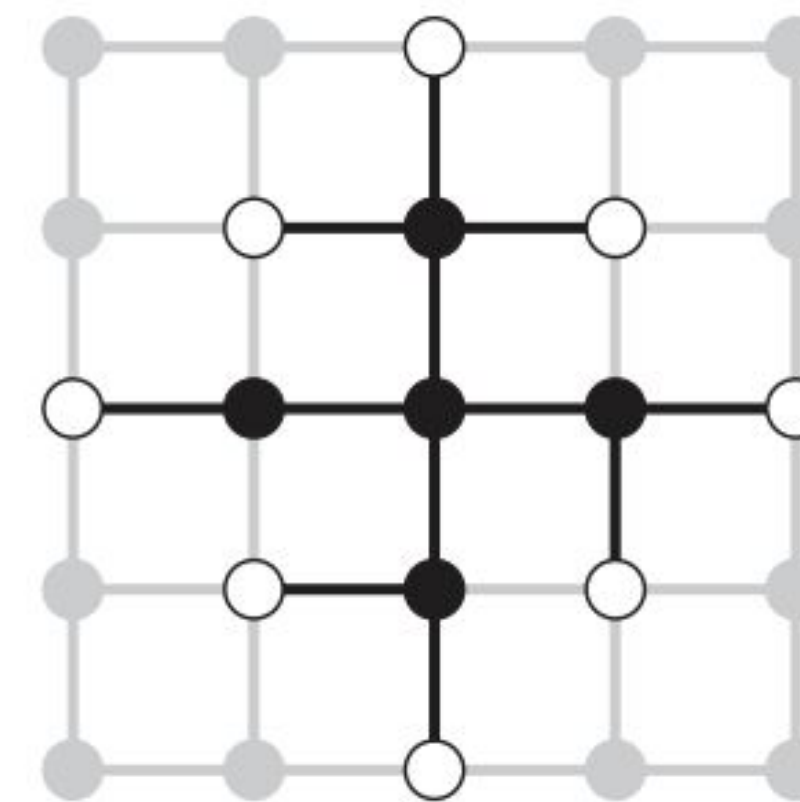
Exterior: nodes not reached (grey nodes)



(a)



(b)



(c)

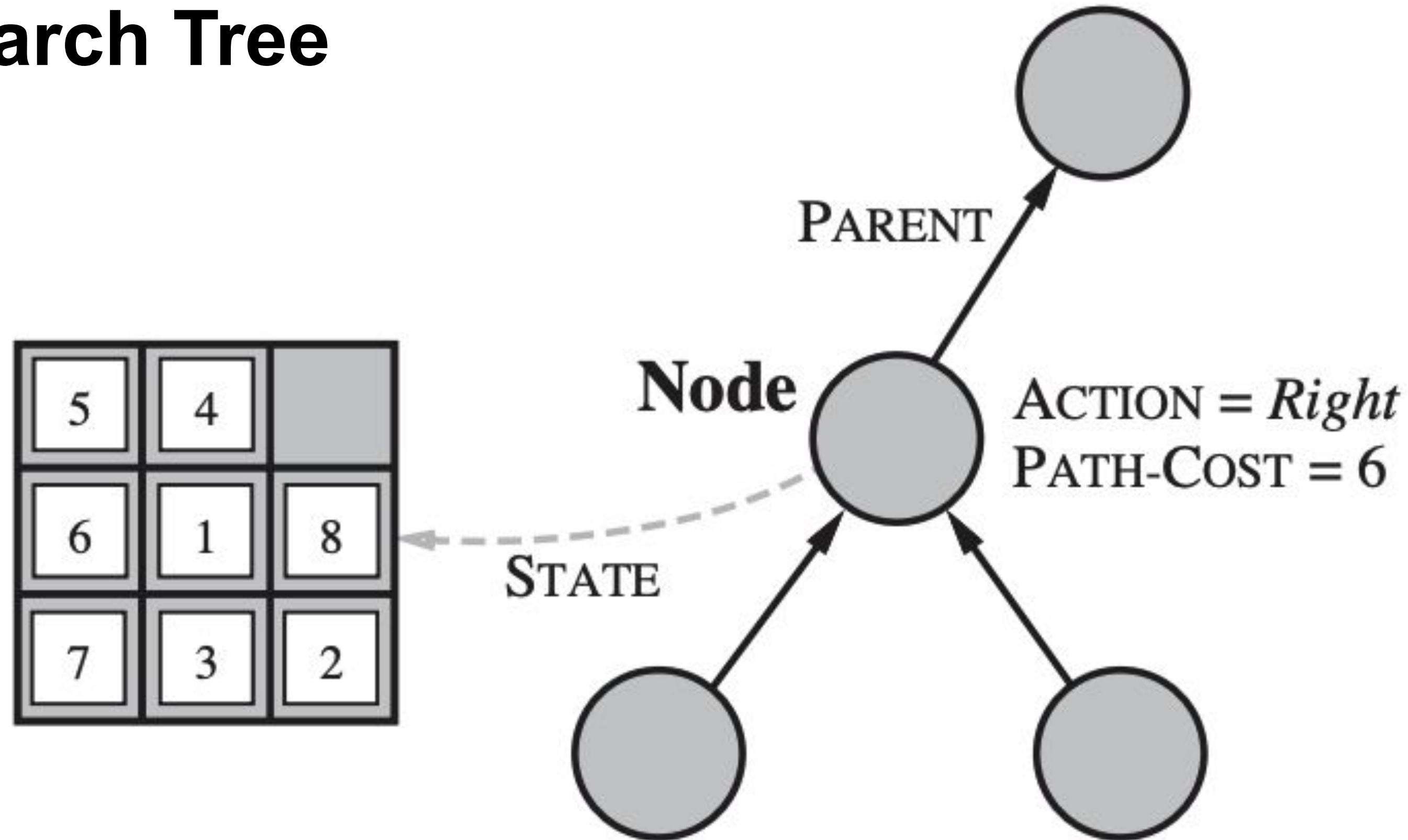
The separation property of GRAPH-SEARCH, illustrated on a rectangular-grid problem. The frontier (white nodes) always separates the explored region of the state space (black nodes) from the unexplored region (gray nodes). In (a), just the root has been expanded. In (b), one leaf node has been expanded. In (c), the remaining successors of the root have been expanded in clockwise order.

Infrastructure for search algorithms

For each node n of the tree, we have a structure that contains four components:

- **$n.STATE$** : the state in the state space to which the node corresponds;
- **$n.PARENT$** : the node in the search tree that generated this node;
- **$n.ACTION$** : the action that was applied to the parent to generate the node;
- **$n.PATH-COST$** : the cost, traditionally denoted by $g(n)$, of the path from the initial state to the node, as indicated by the parent pointers.

Search Tree



function CHILD-NODE(*problem, parent, action*) **returns** a node

return a node with

STATE = *problem.RESULT*(*parent.STATE, action*),

PARENT = *parent*, ACTION = *action*,

PATH-COST = *parent.PATH-COST* + *problem.STEP-COST*(*parent.STATE, action*)

How to select the next node to be expanded?

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

persistent: *seq*, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null

problem, a problem formulation

state $\leftarrow$ UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal $\leftarrow$ FORMULATE-GOAL(*state*)

problem $\leftarrow$ FORMULATE-PROBLEM(*state*, *goal*)

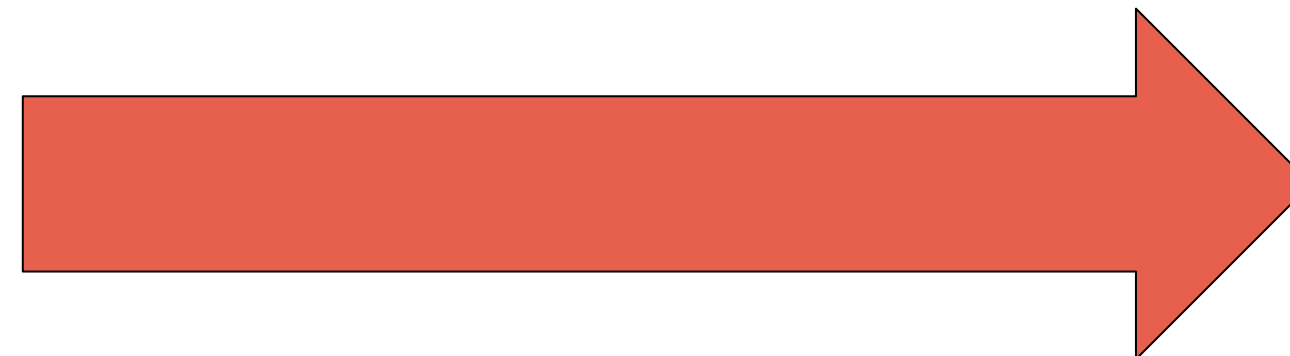
seq $\leftarrow$ SEARCH(*problem*)

if *seq* = *failure* **then return** a null action

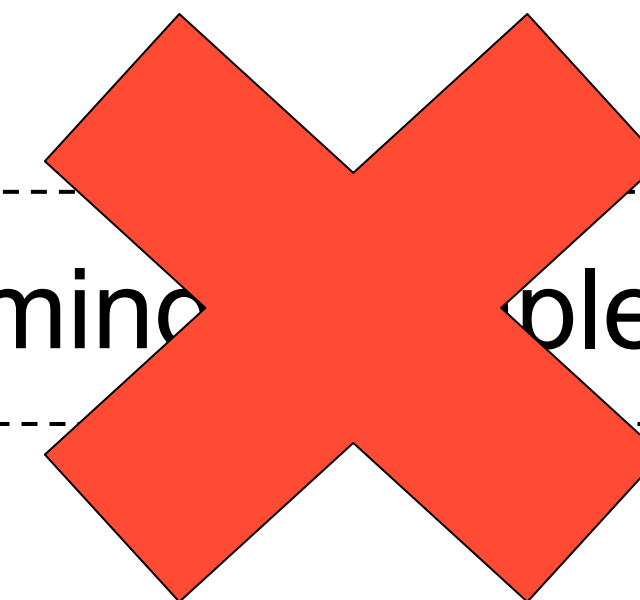
action $\leftarrow$ FIRST(*seq*)

seq $\leftarrow$ REST(*seq*)

return *action*



Assuming a simple queue



How to select the next node to be expanded?

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

persistent: *seq*, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null

problem, a problem formulation

state $\leftarrow$ UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal $\leftarrow$ FORMULATE-GOAL(*state*)

problem $\leftarrow$ FORMULATE-PROBLEM(*state*, *goal*)

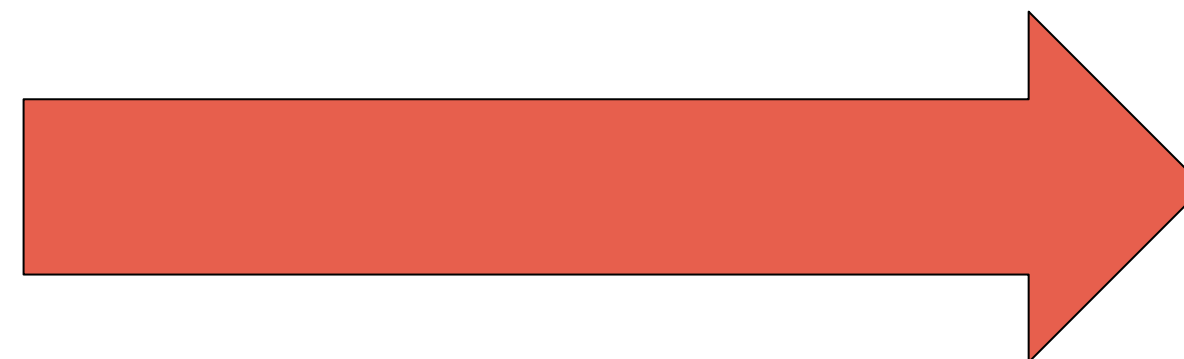
seq $\leftarrow$ SEARCH(*problem*)

if *seq* = *failure* **then return** a null action

action $\leftarrow$ FIRST(*seq*)

seq $\leftarrow$ REST(*seq*)

return *action*



It's a queue:

- Variants: FIFO, LIFO, priority
- Operations: Empty?, POP, INSERT

Search

- Impossibility of exploring all the alternatives in complex problems (brute-force search / exhaustive search)
- If **multiple solutions exist**, usually it suffices to find an **acceptable solution** (not necessarily the optimal/best)
- **Heuristics**: criteria for selecting a promising action/state...
.... ie, what node to be expanded first?

Best first heuristics

Node in the frontier with minimum value of some evaluation function $f(n)$

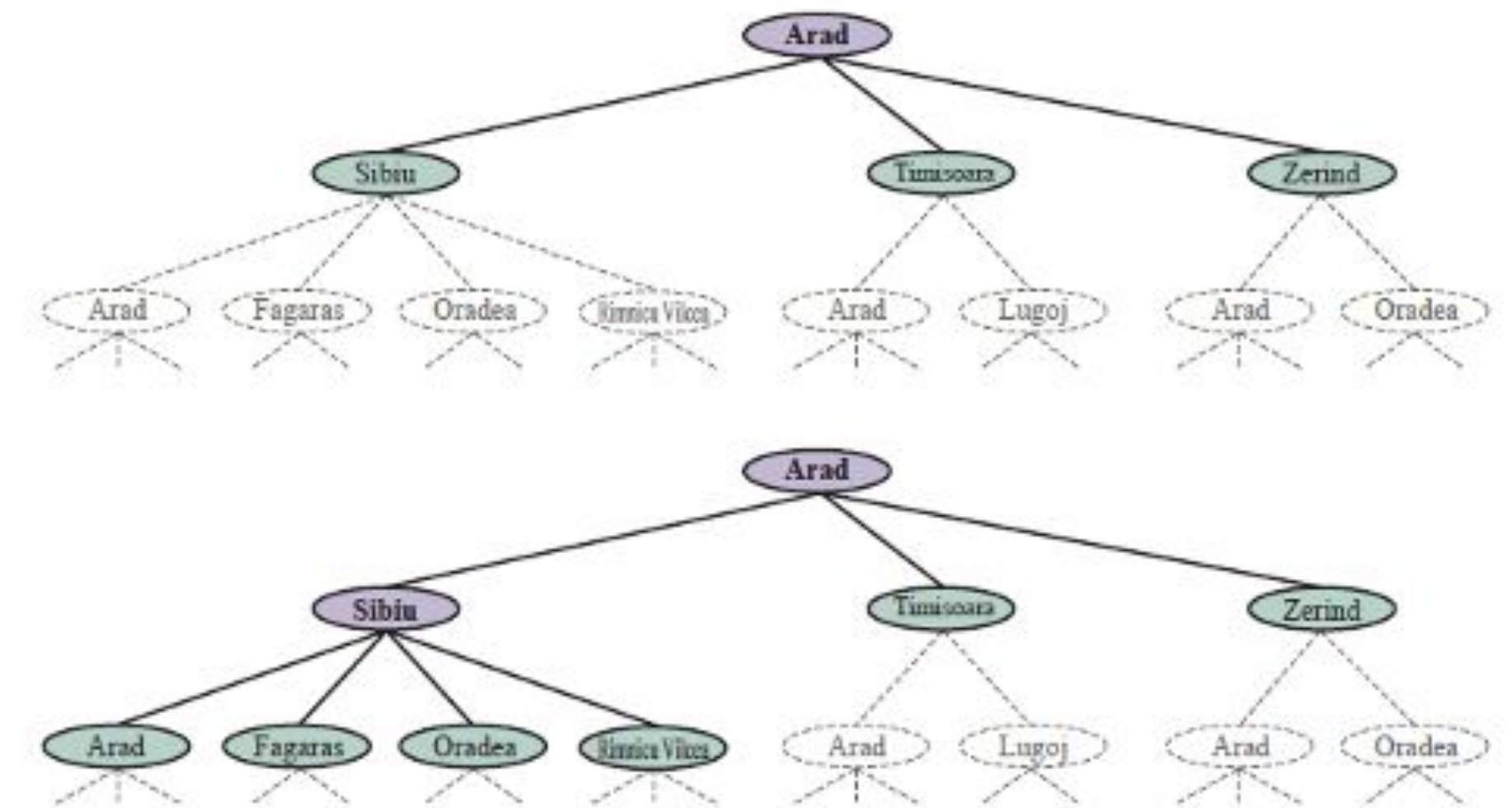
- if goal state, return
- else, expand to generate children nodes

Each child node added to the frontier if

- Not reached before
- Reached before but reached again, now with a lower path cost

Redundant paths, cycles, ...

- “algorithms that cannot remember the past are doomed to repeat it ”



Measurements for selecting search strategies

1. **Completeness:** guarantee to find a solution (failure when not)
 - Need to be systematic
2. **Cost optimality:** find a solution with the lowest path cost of all solutions?
3. **Complexity:**
 - Temporal: how long to find a solution? #states and #actions
 - Spatial: memory needed?
 - Size of the state-space graph (if explicit)
 - Depth (#actions in an optimal solution)
 - Maximum #actions in any path
 - Branching factor (#successors of a node)

Example of performance for a $O(b^d)$ search problem

Each level generates b more nodes in the next level

Assuming $b = 10$ & 1M nodes / second & 1 node / 1000 bytes

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes
Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.			



Real World Implementation of Search Algorithms

neo4j

Building GenAI-Powered Knowledge Graphs for Real-World Impact

Relationships drive business value

Marco De Luca, Neo4j Germany
Principal Solutions Architect

Vienna, 3rd September 2025

Keynote at
SKGi
Workshop at
SEMANTiCS
2025

[Going Meta
by Neo4J](#)



Real World Implementation of Search Algorithms

Available Graph Data Science Algorithms



Pathfinding & Search

- ☐ Delta-Stepping Single-Source
- ☐ Dijkstra's Single-Source
- ☐ Dijkstra Source-Target
- ☐ All Pairs Shortest Path
- ☐ **A* Shortest Path**
- ☐ Yen's K Shortest Path
- ☐ Minimum Weight Spanning Tree
- ☐ Random Walk
- ☐ Breadth & Depth First Search



Centrality

- ☐ Degree Centrality
- ☐ Closeness Centrality
- ☐ Harmonic Centrality
- ☐ Betweenness Centrality & Approx.
- ☐ **PageRank**
- ☐ Personalized PageRank
- ☐ ArticleRank
- ☐ Eigenvector Centrality
- ☐ Hyperlink Induced Topic Search (HITS)
- ☐ Influence Maximization (Greedy, CELF)



Community Detection

- ☐ Weakly Connected Components
- ☐ Strongly Connected Components
- ☐ Label Propagation
- ☐ **Leiden (used for LLM KG Builder)**
- ☐ **Louvain**
- ☐ K-Means Clustering
- ☐ K-1 Coloring
- ☐ Modularity Optimization
- ☐ Speaker Listener Label Propagation
- ☐ Approximate Max K-Cut
- ☐ Triangle Count
- ☐ Local Clustering Coefficient
- ☐ Conductance Metric



Heuristic LP

- ☐ Adamic Adar
- ☐ Common Neighbors
- ☐ Preferential Attachment
- ☐ Resource Allocations
- ☐ Same Community
- ☐ Total Neighbors



Similarity

- ☐ K-Nearest Neighbors (KNN)
- ☐ Filtered K-Nearest Neighbors (KNN)
- ☐ Node Similarity
- ☐ Filtered Node Similarity
- ☐ Similarity Functions



Graph Embeddings

- ☐ Fast Random Projection (FastRP)
- ☐ Node2Vec
- ☐ GraphSAGE

Keynote at
SKGi
Workshop at
SEMANTiCS
2025

[Going Meta
by Neo4J](#)

Contents

Problem solving in AI

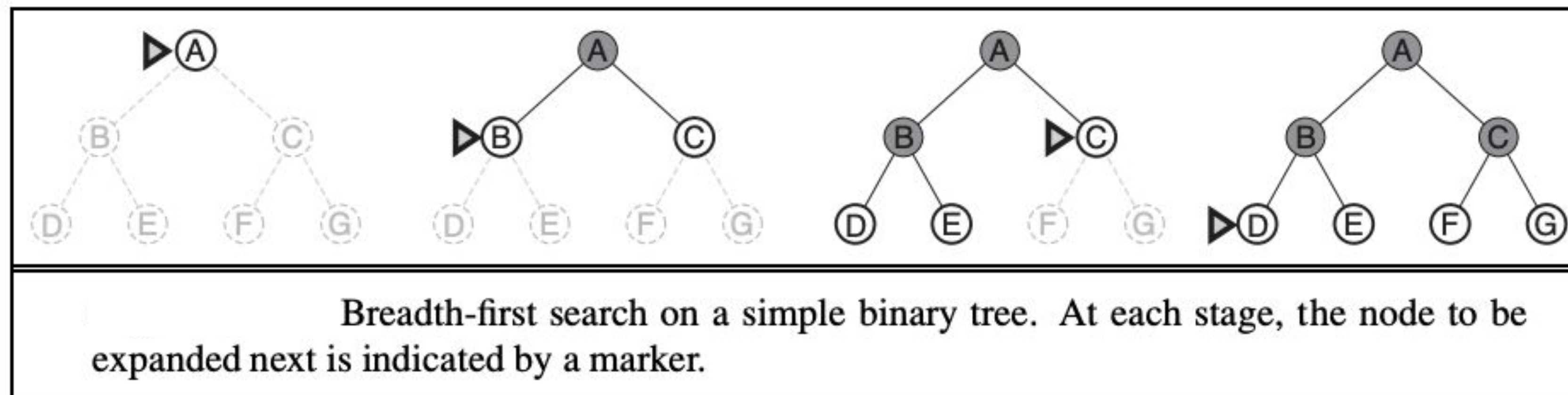
- Search algorithms
- Uninformed search algorithms
- Informed (heuristic) search strategies
- Heuristic functions
- Search for Constraint satisfaction problems

Uninformed search strategies: Breadth-first search

Systematic and complete

Appropriate for same cost actions (cost-optimal) (e.g., cost = 1)

BEST-FIRST with f = node depth



Uninformed search strategies: Breadth-first search

#nodes = $O(b^d)$ at Depth d , with branching factor b

Issue: all nodes in memory

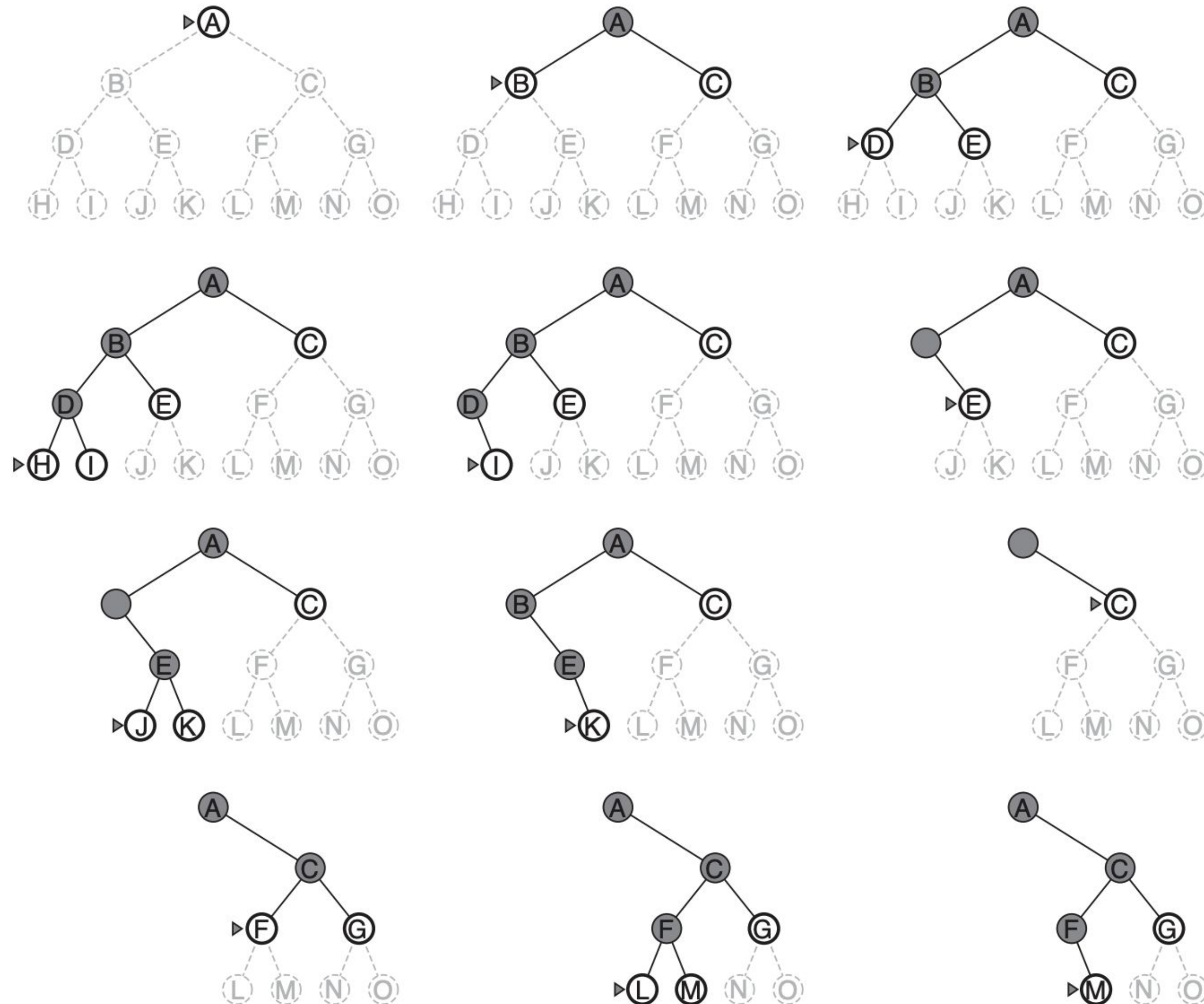
Exponential-complexity search problems cannot be solved (only smallest instances)

29

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
    node  $\leftarrow$  a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    frontier  $\leftarrow$  a FIFO queue with node as the only element
    explored  $\leftarrow$  an empty set
    loop do
        if EMPTY?(frontier) then return failure
        node  $\leftarrow$  POP(frontier) /* chooses the shallowest node in frontier */
        add node.STATE to explored
        for each action in problem.ACTIONS(node.STATE) do
            child  $\leftarrow$  CHILD-NODE(problem, node, action)
            if child.STATE is not in explored or frontier then
                if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
                frontier  $\leftarrow$  INSERT(child, frontier)
```



Uninformed search strategies: Depth-first search



* With M as objective

Uninformed search strategies: Depth-first search

Systematic and complete for finite spaces

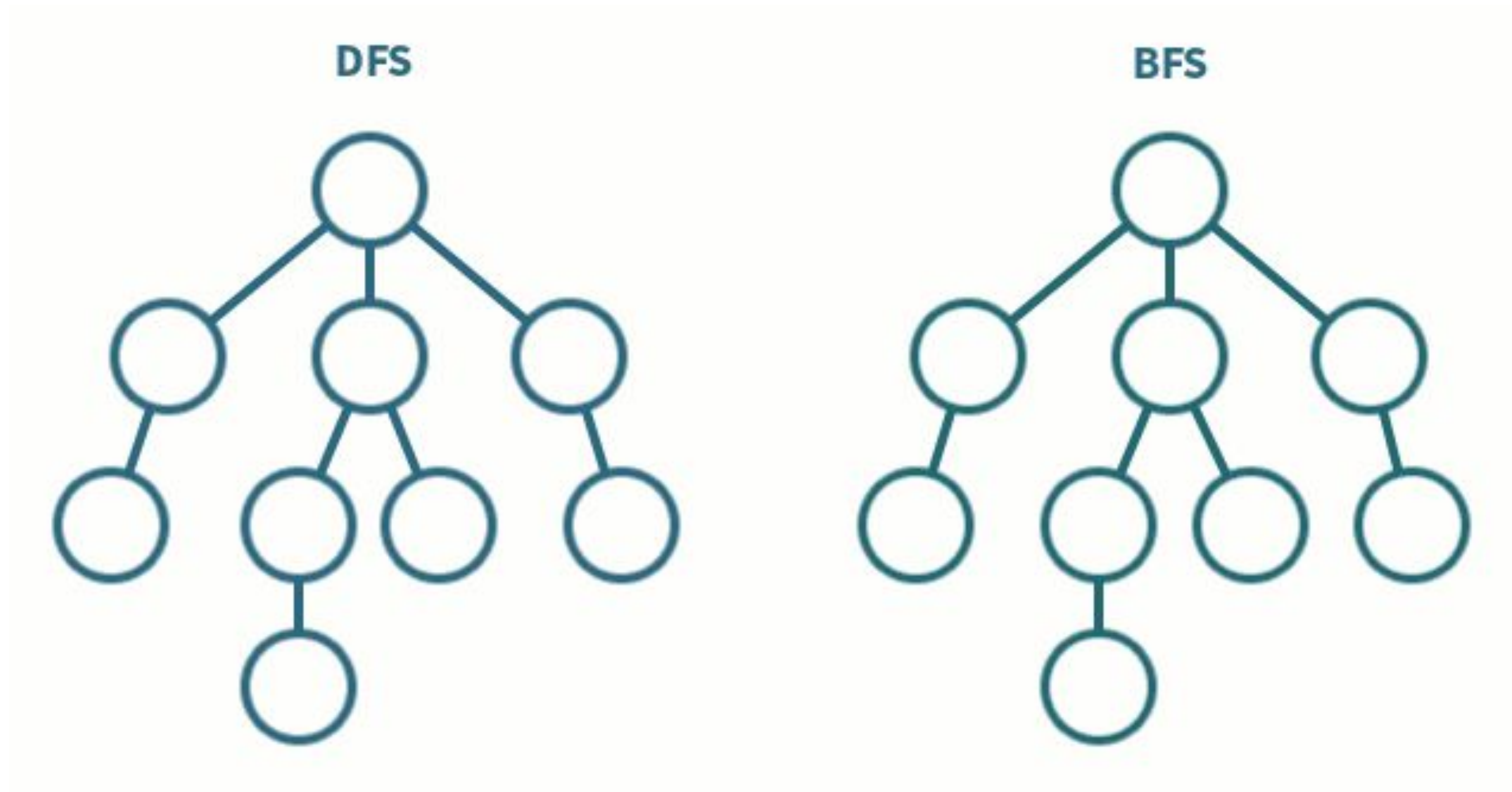
- In cyclic state spaces can get stuck in infinite loops
- In infinite state spaces not systematic=>incomplete

Not cost-optimal (first solution)

BEST-FIRST with $f = \text{node depth}$

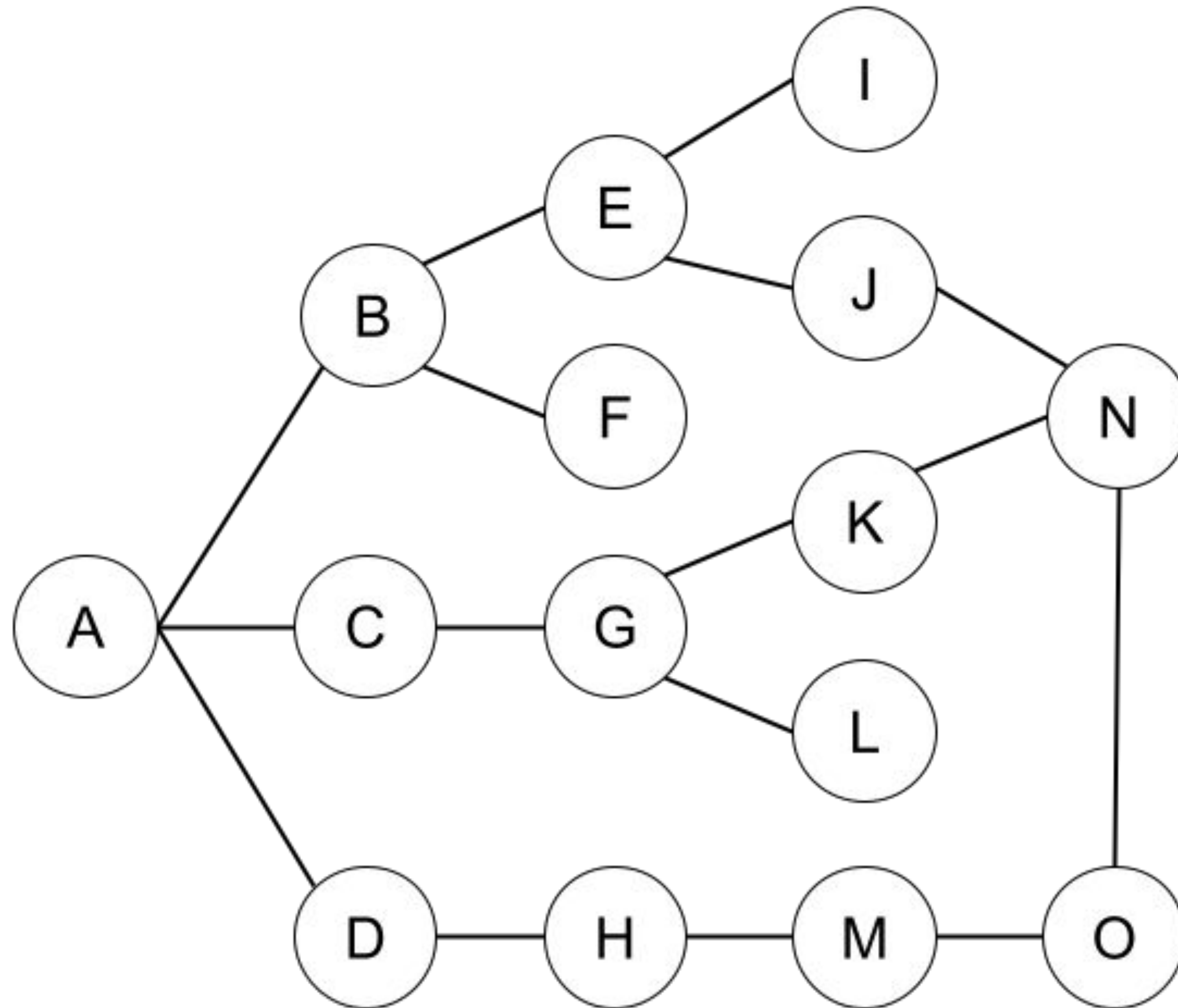
Smaller needs of memory (small frontier): $O(b \cdot m)$

Comparison between BFS and DFS





Exercise 1



Start: A
Goal: N
Cost: 1

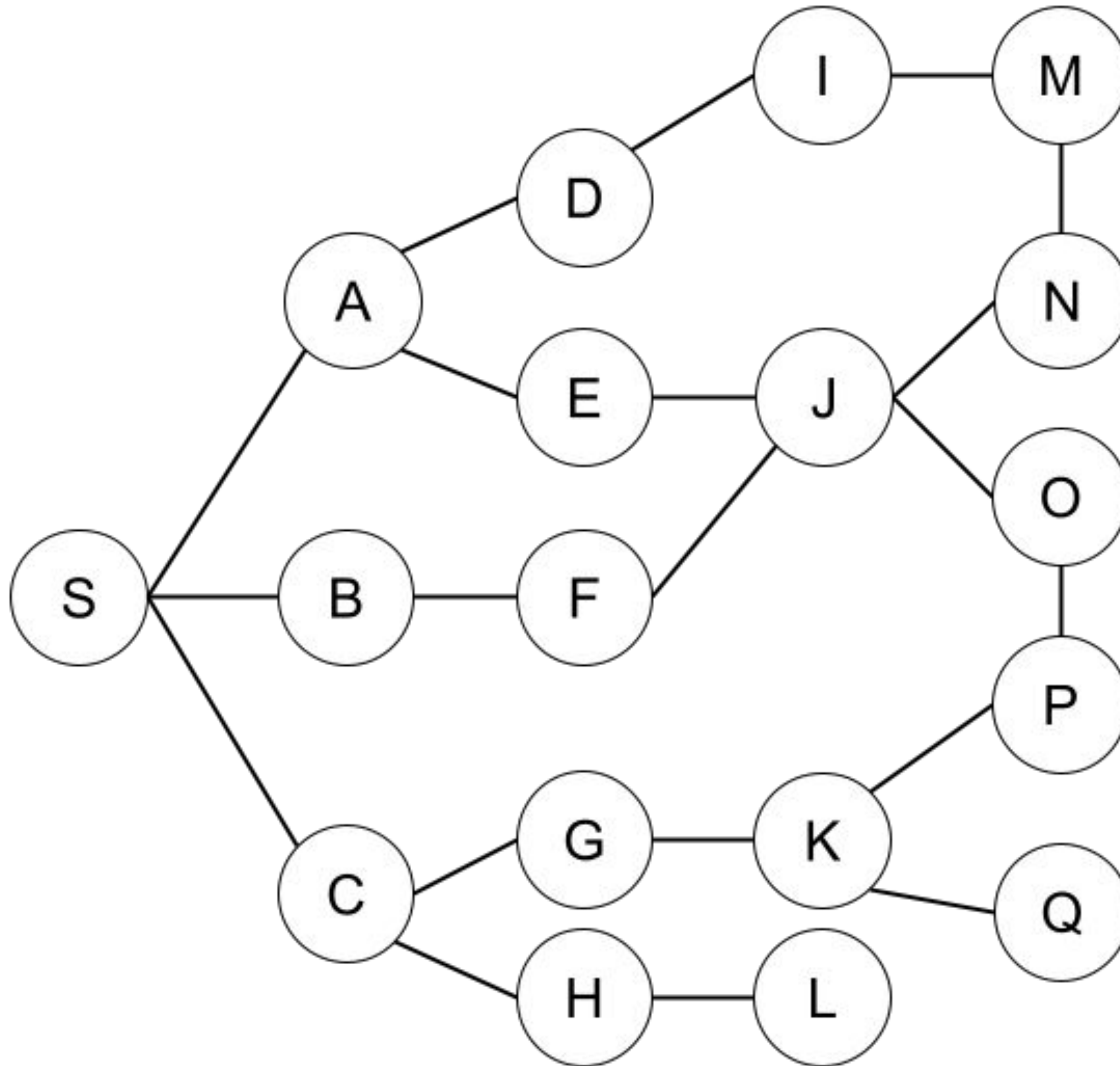
BFS? DFS?

Expanding strategy:
- Alphabetic order



Exercise 2

34



Start: S
Goal: O
Cost: 1

BFS? DFS?

Expanding strategy:
- Alphabetic order

Uninformed search strategies: Depth-limited search

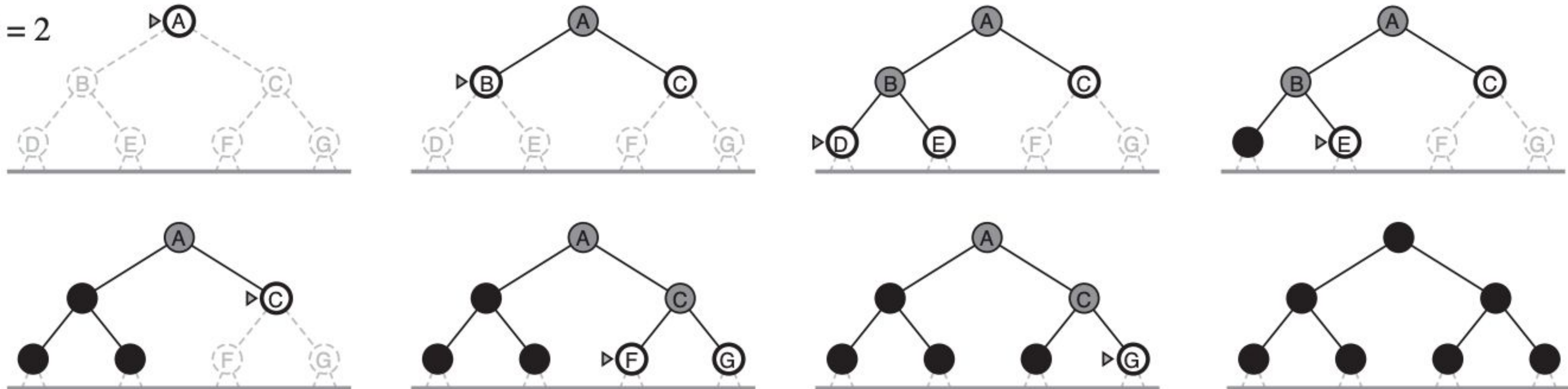
Limit = 0



Limit = 1

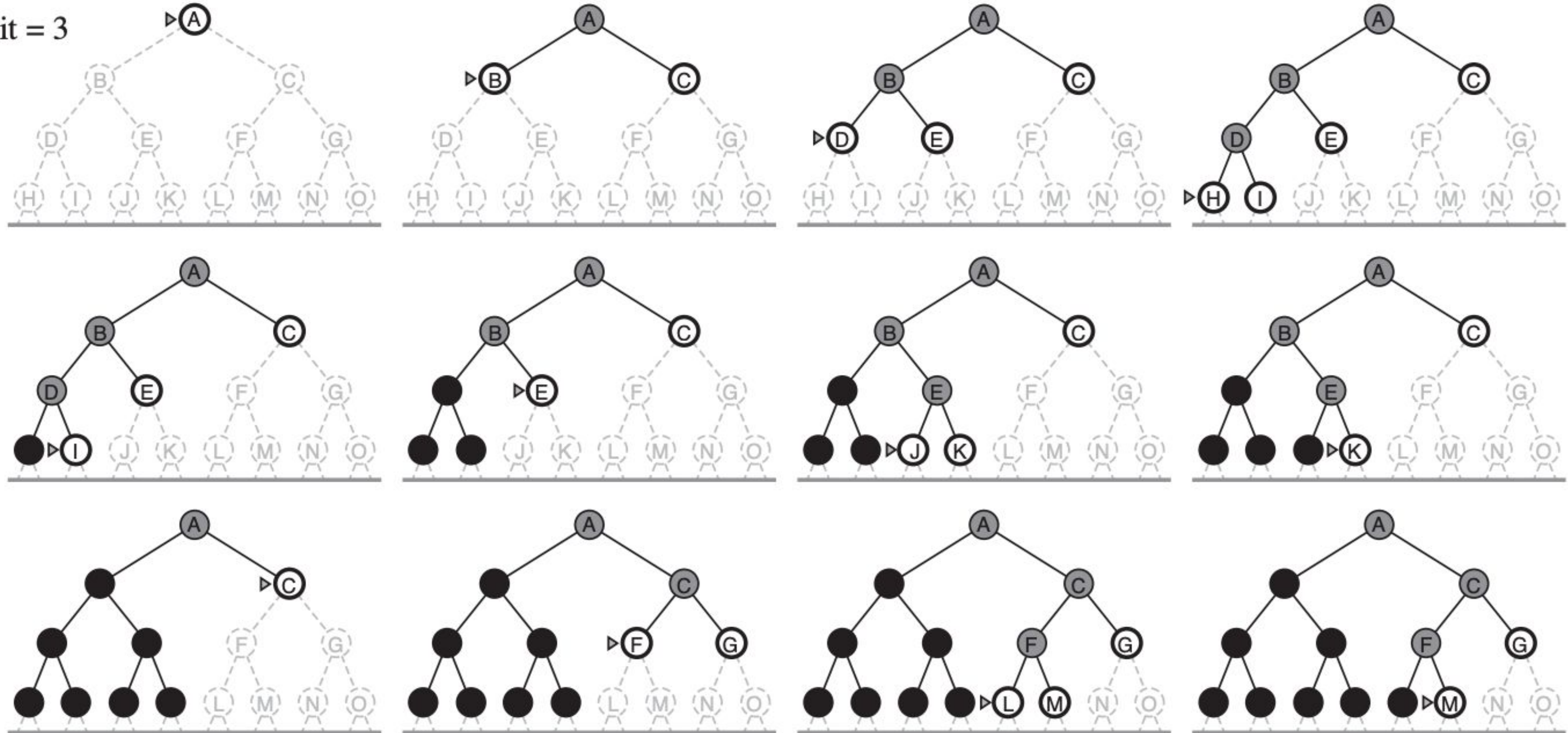


Limit = 2

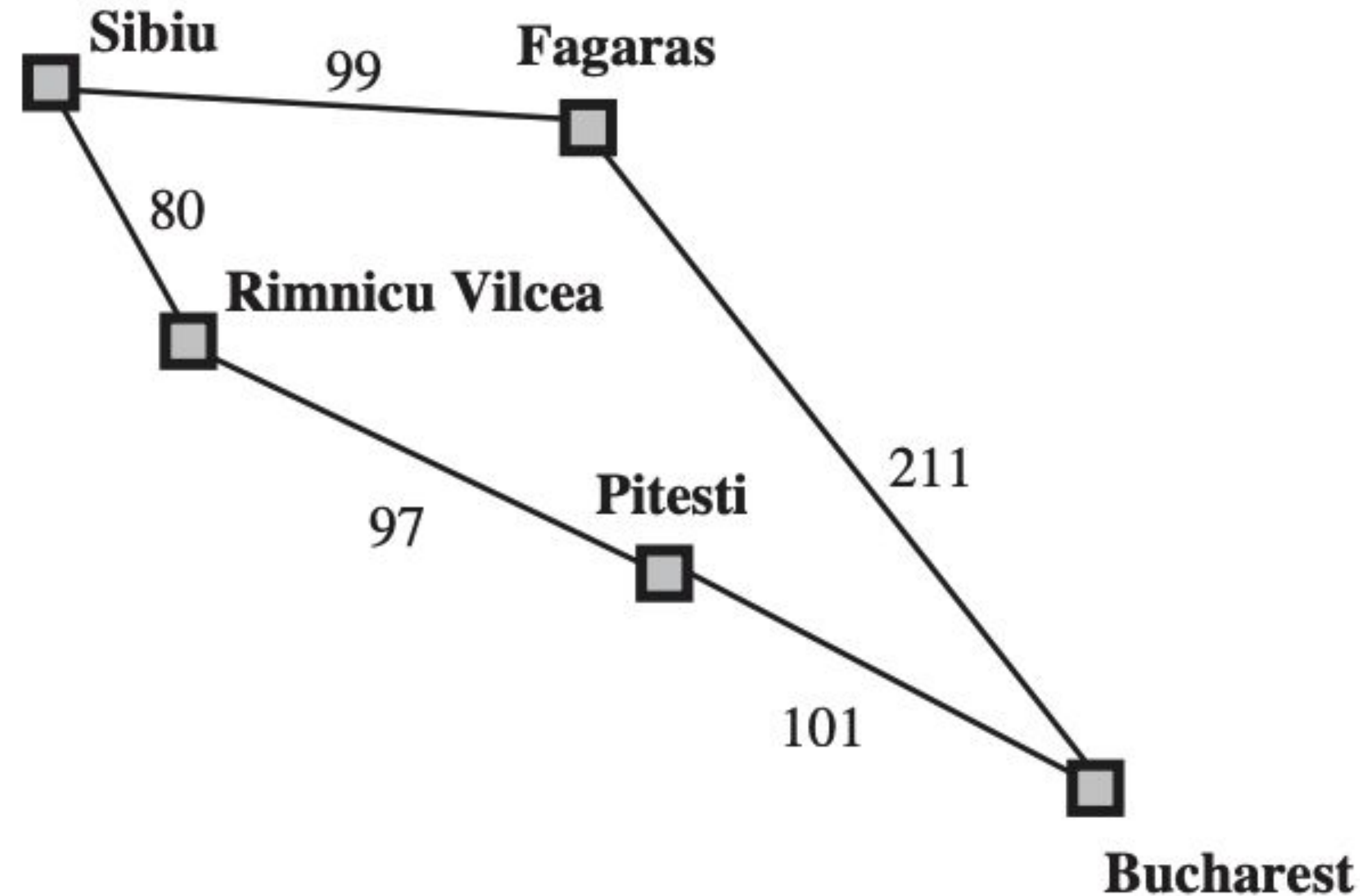


Uninformed search strategies: Depth-limited search

Limit = 3



Uninformed search strategies: Uniform-cost search



$S(0)$
 SR (80) / SF (99)
 SRP (177=80+97) / SF (99)
 SRP (177=80+97) / SFB (310=99+211)
SRPB (278=80+97+101) / SFB (310=99+211)

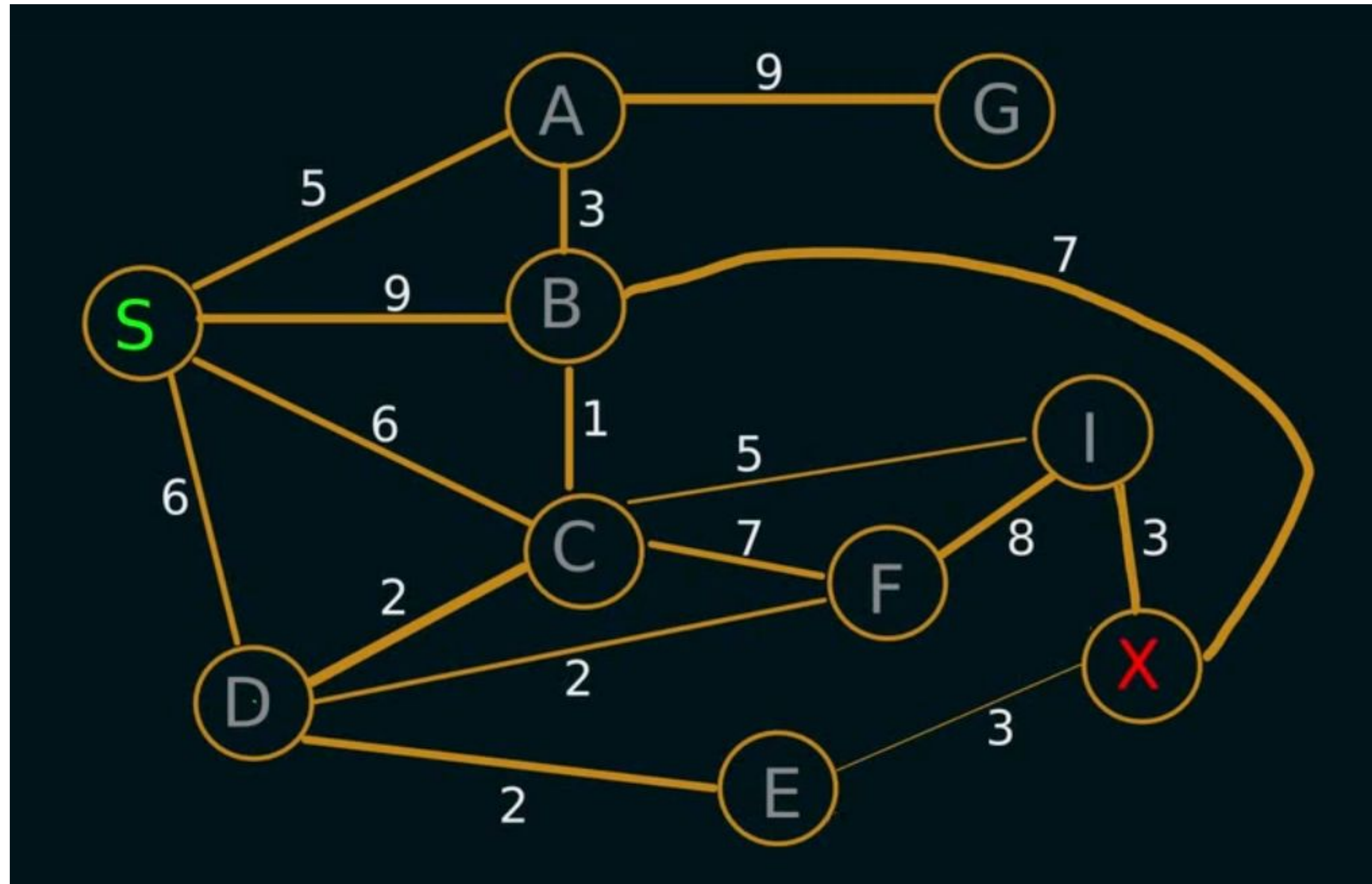
The cost for each path is different (breadth-first cost is equal)

BEST-FIRST with **f=path cost**

Instead of expanding shallowest node,

it expands the node with the lowest path total cost

Exercise: Uniform cost search



Exercise 2: Search the solution following the uniform cost search algorithm

Costs of uninformed search algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}
Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; ℓ is the depth limit. Superscript caveats are as follows: ^a complete if b is finite; ^b complete if step costs $\geq \epsilon$ for positive ϵ; ^c optimal if step costs are all identical; ^d if both directions use breadth-first search.						

Resolving Problems by Searching

AIF-MSc on AI / 2025-2026

David Chaves-Fraga

University of Santiago de Compostela (Spain)

david.chaves@usc.es



Singular Research Center on
Intelligent technologies